Department of Computer Science, University College London

COMP0227 Lab 02: Semantic Segmentation in Images and in SLAM

13th October, 2026

1 Scope and Purpose

In Lecture 01, we reviewed per-pixel properties of a scene and heavily focused on closed-world *semantic segmentation*: every pixel in the scene is given a class label such as *road*, *car* or *sky*. pySLAM supports multiple segmentation algorithms. In this lab, we'll study the performance of these algorithms and see what they can contribute to SLAM.

This lab has three parts:

1. Run the segmentation networks on *single* images and compare them. Which work, what do they get wrong, and how long do they take on your computer? At the end of this part you should be able to choose one for the rest of the lab.
2. Run one method over a whole video, the KITTI sequence of Lab 00, and look at how stable the labels are from one frame to the next.
3. Use the labels in SLAM: mask out the pixels of the things which move, so that no features are detected on them, and measure whether this helps.

This lab assumes that you have completed Labs 00 and 01, and that pySLAM is installed and working. As before, run all the commands from the `pyslam` folder.

2 Setting Up

The lab is built around DeepLabV3. This is slightly simpler than DeepLabV3+ described in the lectures. We are using this simpler model because it needs nothing beyond the default level of pySLAM: it downloads its weights (160 MB) the first time it runs. Everything below can be done with it alone.

SegFormer, which is better on street scenes, is available in the `semantics` pixi environment. It is installed by the first command which uses it, for example:

```
pixi run -e semantics semantic-segmentation --method SEGFORMER --test kitti
```

Note that this will install the `semantics` pixi environment. This is about 6GB[1]. The first time you run this, the code will download the SegFormer weights (about 30MB). If you do not have a GPU on your machine, under Linux or WSL2 you can run:

```
pixi run -e semantics-cpu semantic-segmentation --method SEGFORMER --test kitti
```

However, this will obviously be much slower.

If disk space is short, `pixi clean cache` deletes the copy of the downloaded packages which pixi keeps; it downloads them again if it ever needs them.

pySLAM also supports CLIP. It needs one more command first, which installs its code and downloads the models of all the methods of the `semantics` level, CLIP's among them (about 2 GB and a few minutes; with an NVIDIA GPU about 13 GB and a quarter of an hour, because it also gets two methods that only run there):

```
pixi run -e semantics models-semantics
pixi run -e semantics semantic-segmentation --method CLIP --test kitti
```

However, since CLIP is both very large and performs very poorly on semantic segmentation tasks, we recommend you consider it to be optional and only try it if you have a lot of storage, a lot of compute capacity and fancy the challenge.

On native Windows

If you installed pySLAM on Windows itself rather than in WSL2 (the experimental route of the installation guide), four things differ.

- **Add `-e default-win` to every `pixi run` command of this sheet, as in the guide, for example `pixi run -e default-win semantic-segmentation --test kitti`.**
- **Only DeepLabV3 is available. SegFormer and CLIP need the `semantics` level, which does not exist on native Windows: borrow a neighbour's numbers for them, or use WSL2. DeepLabV3 runs on the CPU, about 3 s per image on a fast laptop, so Part 2 with `--step 5` takes about 11 minutes (use `--step 10` on a slower one).**
- **The commands are written for the shell of Linux, macOS and WSL2. In PowerShell, set the folder of Activity 7 with `$S = "C:\path\to\rgbd_dataset_freiburg3_walking_halfsphere"`, type each command on one line (without the `\` at the ends of the lines), and write the associations file with `| Out-File -Encoding ascii $S/associations.txt` instead of `> $S/associations.txt`: PowerShell's `>` writes the file as UTF-16, which pySLAM cannot read.**
- **The evaluation of Activities 7 and 8 does not write its `report.html` on native Windows yet (it stops with an error about `applymap`): read the tables it prints at the end, or the `table_*.csv` files in the same folder.**

3 The Methods

Table 1 lists the methods. The names are the ones pySLAM uses.

[1] This is thanks to the joys of Python where a separate environment has to be installed to stop the dependencies from destroying one another.

Name	Network	Gives	Classes
DEEPLABV3	DeepLabV3, ResNet-50	a class for every pixel	21 object classes (Pascal VOC)
SEGFORMER	SegFormer-B0, a transformer	a class for every pixel	19 street classes (Cityscapes)
CLIP	CLIP ViT-L/14	a class for every pixel, from text	the 19 Cityscapes class names

Table 1: The segmentation networks of this lab. DEEPLABV3 **runs at the default level; the other two need the** semantics **level. pySLAM has others, including object detectors, which are not used here.**

Because we are using closed-world models, each network only knows the classes it was trained on. DeepLabV3 was trained on photographs of objects (Pascal VOC: person, car, bus, bicycle, cat, chair, . . .): everything else is *background*. SegFormer was trained on street scenes (Cityscapes: road, building, vegetation, sky, car, person, . . .) and has no class for a desk or a chair. A full list of classes for the different classifiers can be found in pyslam/semantics/semantic_labels.py**.**

The full list of methods, with what each gives, is printed by:

```
pixi run semantic-segmentation --list
```

4 Part 1: Single Images

Activity 1: Segment One Image

Run:

```
pixi run semantic-segmentation --method DEEPLABV3 --test kitti
```

This segments one frame of the KITTI video with DeepLabV3. The first run of the network will be slow, because the model is loaded (and, with some networks, compiled for your GPU). Three windows will open:

- Image**: the image.**
- Classes**: the class of every pixel, in the colours of the method, with the largest classes and their shares of the image underneath.**
- Classes over the image**: the same colours drawn over the image, so that you can see what each region is.**

An example is shown in Figure 1. Press a key in any window to close the program. The last line in the terminal is a summary:

```
Summary: DEEPLABV3 on kitti (1226x370 pixels): 21 classes (Pascal VOC) on mps;
1.57 s per image (mean of 3 runs after the first, which took 1.90 s; 1.9 s to
load the model); background 99%, car 1%, bicycle <1%
```

Your numbers, especially the times, will be different.

- **Almost everything is *background*. Look at the list of DeepLabV3's classes (Pascal VOC): what in this street could it have labelled at all? What did it find?**
- **Look at the outlines of the cars. Are they tight? Look at the car in the distance, and at the bicycle on the right.**
- **The image is 1226 pixels wide, but the network sees it squeezed to 512×512 and predicts a class for every 8×8 block of that. Can you see this in the outlines?**



Figure 1: The three windows of Activity 1, with `DEEPLABV3` on `kitti`: the image, the class of every pixel with the largest classes underneath, and the classes over the image.

Activity 2: Compare the Networks on the Same Image

If you have the `semantics` level, run the same image through SegFormer, and optionally through CLIP:

```
pixi run -e semantics semantic-segmentation --method SEGFORMER --test kitti
pixi run -e semantics semantic-segmentation --method CLIP --test kitti
```

and fill in the table from the summary lines. If you do not, borrow a neighbour's numbers for the other rows: the comparison is the point.

	Largest classes	Time per image (s)	Load (s)
DEEPLABV3			
SEGFORMER			
CLIP			

- **SegFormer labels the road, the buildings, the trees and the sky, which DeepLabV3 called background. Is DeepLabV3 a worse network, or a different one? Look at the boundaries SegFormer draws: where are they sharp, and where ragged (poles, tree trunks, the far end of the road)?**
- **(Optional). CLIP is not a segmentation network: it matches each patch of the image to the text of the class names. How does its result compare with SegFormer's, which was trained on the same classes?**
- **Which is the fastest, and which the slowest? The KITTI video has 10 frames per second. Which of them could keep up with it on your computer?**
- **Compare your times with your neighbours'. The summary line says which device the network ran on. How large is the difference between a GPU and a CPU?**
- **If you had to choose one network to label every frame of a street video on your computer, which would it be, and why?**

Activity 3: Greyscale and Other Images

Many robotics datasets, such as EuRoC and TUM, are black and white because, historically, they used machine vision cameras[2] However, the segmentation networks have been trained on colour images. Therefore, one question is: does this actually matter? To test this, the option `--grey` turns the image to greyscale before it is segmented:

```
pixi run semantic-segmentation --method DEEPLABV3 --test kitti --grey
pixi run semantic-segmentation --method DEEPLABV3 --test desk --grey
```

- **Compare the shares of the classes with and without `--grey`, on `kitti` and on `desk` (Table 2). Does DeepLabV3 mind?**
- **If you have SegFormer, try it on `kitti` with `--grey`. Which classes change most? Look at the sky, the vegetation and the road: what has the network lost, and why would one network mind more than the other?**

Table 2 lists all the test images. Use `--test` to choose one, or `--image` with an image file of your own.

- **Run DeepLabV3 on `desk`, then on `mars`. What does it find on the desk, and what does it miss? On Mars it says *background* everywhere: is that right, or just lucky?**
- **If you have SegFormer, run it on `desk` and on `mars`. It has to give every pixel one of its 19 street classes. What does it choose, and how confident does the picture look? A segmentation network has no way to say "I do not know".**
- **Take a photograph of a street, or of your room, with your phone, and segment it with `--image`. Which network did best on it?**

[2] KITTI is unusual because two of its four cameras are in colour.

Name	The image
`kitti`	a frame of the KITTI 06 video (a street)
`kitti_next`	the frame half a second later
`desk`	a desk with objects on it (indoors)
`churchill`	a building
`box`	a box among other objects
`mars`	the surface of Mars

Table 2: The test images

5 Part 2: A Whole Sequence

Activity 4: Watch the Labels over Time

The same script segments every frame of a video. Run it on the colour KITTI 06 video which comes with pySLAM, and save the label maps for Part 3:

```
pixi run semantic-segmentation --method DEEPLABV3 \
    --video data/videos/kitti06/video_color.mp4 --save labels/kitti06 --step 5
```

The three windows now show the frames as they are segmented. The video has 1101 frames; `--step 5` **segments every fifth one, which is enough for Part 3, and takes 220 times your time per image from Activity 1: about five minutes on a laptop GPU, longer on a CPU. Leave the** `--step` **out if your computer is fast, or if you use SegFormer (the same command with** `-e semantics` **and** `--method SEGFORMER`**, and another** `--save` **folder). Press** `p` **in a window to pause, and** `p` **again to carry on;** `q` **or Esc stops.**

While it runs, watch the `Classes over the image` **window. The questions here are about what you see, so pause when something catches your eye.**

- **Watch a parked car as the camera approaches and passes it. Is it a car all the way? Does its outline keep up with it? When does the network first see it, and when does it lose it?**
- **Do the labels flicker: a car in one frame, background in the next? Where does this happen: on large cars, small ones, cars at the edge of the image?**
- **Watch what happens in shadow, and when the car turns and the scene changes quickly. Where are the labels least reliable?**
- **With SegFormer, the road, the buildings and the trees are labelled too. Do the labels of those large regions stay the same from one frame to the next, or do they flicker, and where: inside the regions, or at their boundaries?**
- **Would you trust the label of one frame for a pixel, or would you want to look at several frames? SLAM, which tracks points over many frames, could do this.**

When it finishes, the summary line gives the mean shares of the classes over the sequence, the mean time per frame, and one more number: the fraction of the pixels whose label changed from one segmented frame to the next, on average.

Activity 5: Measure the Stability

Watching is not a measurement. The folder `labels/kitti06` **now holds, besides the label maps, a file** `shares.csv` **with one line per segmented frame: the frame number, the time it took, the fraction of the pixels whose label changed (**`changed_fraction`**), and the share of every class. Open it in a spreadsheet, or in Python:**

```
import pandas as pd
shares = pd.read_csv("labels/kitti06/shares.csv")
shares.plot(x="frame", y=["background", "car", "person", "bicycle"])
shares.plot(x="frame", y="changed_fraction")
```

(`pixi run python` **opens a Python with pandas. SegFormer's file has no** `background`**: plot** `road`**,** `building`**,** `vegetation` **and** `sky` **in its place.)**

- **Look at the share of** `car` **in the first plot. Cars come and go as the camera passes them, so it rises and falls; but does it also jump from one segmented frame to the next? A jump is a sign of flickering, because the scene itself changes little between them (half a second apart with** `--step 5`**).**
- **In the second plot, find the frames where** `changed_fraction` **is largest and look at them in the video (the frame number is in the title of the** `Image` **window;** `p` **pauses). Is the network flickering, or is the scene really changing?**
- **Two things make a pixel change its label: the camera moves, so a different part of the scene is at that pixel, and the network changes its mind. Can you separate the two from this file? What would you need to measure the second on its own?**
- **Which classes of the "things that move" (person, car, bus, bicycle, motorbike, train) appear in this sequence, and for what share of the pixels? Are there people in KITTI 06? If the network says so, where?**

If you have SegFormer, segment the sequence with it too (another `--save` **folder), and compare the two** `shares.csv` **files.**

- **Do the two networks agree about where the cars are? Is the faster network also the more stable?**

6 Part 3: Masking Moving Objects in SLAM

SLAM assumes the world is still: a map point is a fixed place, and tracking it tells the camera where it is. A feature on a moving car breaks this assumption. The usual answer is to detect such points geometrically, as outliers, after the event. With semantics we can do better: never detect features on the pixels of things which move.

pySLAM's feature detectors already accept a *mask*: an image with one value per pixel, in which keypoints are only kept where the mask is not zero. What is missing is the mask. You will write it.

Activity 6: Write the Mask

Open the file `pyslam/semantics/semantic_mask.py`**. It contains the function**

```
def mask_from_labels(labels, class_names, masked_classes, dilation=0):
```

which, as supplied, returns `None`: no mask. Its docstring says what it is given and what it must return. In short: it gets the label map of one frame (an integer array, one class per pixel), a dictionary from the labels to the names of the classes, the names of the classes to mask, and a number of pixels; it must return an array of the same shape, of type `np.uint8`, which is 0 on the pixels of the masked classes, grown outwards by `dilation` pixels, and 255 everywhere else.

Write it. You need about ten lines. Useful functions: `np.isin` tests which pixels have one of a list of labels; `cv2.dilate` with `cv2.getStructuringElement` grows a region; `np.where` picks between two values.

Then run SLAM with the label maps you saved in Part 2:

```
pixi run slam --semantic-mask labels/kitti06
```

This runs SLAM on the KITTI 06 video, as in Lab 00, and for every frame reads your label map, calls your function, and gives the mask to the feature detector. In the `Camera` window the masked regions are darkened. By default the masked classes are the things which move in a street scene (person, car, bus, bicycle, motorbike, train, and with SegFormer also rider, truck and motorcycle; a name the network does not have is ignored). `--mask-classes` changes them, and `--mask-dilation` grows the regions:

```
pixi run slam --semantic-mask labels/kitti06 --mask-classes car bus \
    --mask-dilation 10
```

Where the label maps were saved every fifth frame, the frames in between use the last map saved before them.

At the end, before the trajectory error, it prints how many frames were masked and what fraction of the pixels the mask covered. If your function still returns `None`, a warning says so and SLAM runs without a mask.

- **`Camera`: are there still features on the cars? Look at their edges. What does `--mask-dilation` change?**
- **`# matches`: how does the number of matches compare with Lab 00? The detector still looks for 2000 features, but some of them are now thrown away.**
- **KITTI 06 is a quiet suburb. Most of the cars in it are parked. Is masking a parked car a good idea? What does SLAM lose?**
- **What happens if you mask `background` instead, with `--mask-classes background`, so that only the cars and the bicycles are left? Does SLAM survive? With SegFormer's maps, try masking `building` and `road`.**

Activity 7: People in the Picture

KITTI 06 has almost nothing moving in it. The TUM RGB-D sequence *freiburg3 walking halfsphere* (its folder is called `rgbd_dataset_freiburg3_walking_halfsphere`) is a hand-held camera in an office, with two people walking about in front of it and often filling a third of the picture: the case the mask was made for. It is one of the "dynamic objects" sequences of the TUM RGB-D dataset; the others (`walking_xyz`, `walking_static`, `walking_rpy` and the `sitting` ones) work the same way. Download it (about 1 GB) from the TUM RGB-D web site and unpack it.

The sequence comes with depth images. SLAM can use them once it knows which depth image goes with each colour image. This command writes a file that pairs them (run it from the `pyslam` folder, with the sequence wherever you unpacked it):

```
S=path/to/rgbd_dataset_freiburg3_walking_halfsphere
pixi run python scripts/associate.py $S/rgb.txt $S/depth.txt > $S/associations.txt
```

(`S` is just a shorthand for the folder; the commands below use it too. In PowerShell on native Windows, see the end of Setting Up.)

Then segment it. Do this after the file above exists, so that the frame numbers of the label maps agree with the frames SLAM will read:

```
pixi run semantic-segmentation --method DEEPLABV3 --tum $S --save labels/halfsphere
```

DeepLabV3 is the right network here: its classes include *person* and it was trained on photographs rather than street scenes (SegFormer finds the people too, but calls the office a building and the desks trains). There are 1021 frames; add `--step 5` on a slow machine.

- **Watch the labels. Are the people found in every frame? What happens when they are close to the camera, or partly out of the picture?**

1. With depth

Run SLAM on it, masking the people. With the associations file in place, `--tum` uses the depth images (RGB-D SLAM: the scale of the map is known, and every feature has a depth from the start).

```
pixi run slam --tum $S --settings settings/TUM3.yaml \
    --semantic-mask labels/halfsphere --mask-classes person --mask-dilation 10
```

Then measure it, with and without the mask:

```
pixi run slam-evaluation --tum $S --settings settings/TUM3.yaml \
    --semantic-mask labels/halfsphere --mask-classes person --mask-dilation 10 \
    --runs 2
```

The sequence is 30 seconds long, so this is four runs of a minute or two. The report (`report.html` in the folder `results/eval_<date>`) has two columns, `ORB2` and `ORB2+mask`, with the tables of Lab 01: `rmse`, `max`, `percent_tracked` and `percent_lost`.

- **Compare the errors with and without the mask. By what factor do they differ? Is the difference larger than the difference between runs?**
- **Look at the trajectory plots in the results folder (one per run). What did the people do to the unmasked runs? Where in the room is the estimate wrong?**
- **Try `--mask-dilation 20`. Why might a larger margin be needed with a walking person than with a parked car?**

2. Without depth

Now the same sequence as a monocular camera, as in Lab 01: add `--sensor mono` to the two commands of part 1 (the same label maps serve; only the depth images are left unused). The evaluation becomes:

```
pixi run slam-evaluation --tum $S --sensor mono --settings settings/TUM3.yaml \
    --semantic-mask labels/halfsphere --mask-classes person --mask-dilation 10 \
    --runs 2
```

- **On most computers the mask now makes the result worse, not better, and in every run; compare with your neighbours, because on some it does not. Find out where in a run that got worse it goes wrong (the trajectory files and plots help) and why. What does a monocular system have to do at the start that an RGB-D system does not, and what were the people contributing to it?**

- **Add `--mask-after-init`, which only masks while tracking is OK: the initialisation, and relocalisation after a loss, keep every keypoint. Does it cure it? Why would the first frames matter so much more than the rest?**
- **Do the same with depth: run the evaluation of part 1 again with `--mask-after-init`. Does it change anything there? Why, or why not?**
- **`--mask-after-init` is a blunt fix: it trusts the moving things at the start and distrusts them ever after. Can you think of a more elegant way of using the features on things which can move, for as long as they do not?**

Activity 8: Back to the Street

As in Lab 01, one run is not a measurement. Run the evaluation on KITTI 06 with the label maps of Part 2, with and without the mask:

```
pixi run slam-evaluation --semantic-mask labels/kitti06 --runs 3
```

This is six runs of about two minutes each. `--mask-classes` and `--mask-dilation` work here too, and `--features` adds other feature sets (each one is run with and without the mask).

- **It makes little difference. Did you expect this, after Activity 7? From Activity 5 you know what share of the pixels the moving classes cover in this sequence; what else differs between a quiet suburb and an office with two people walking about?**
- **Think about what the mask removes from the map, and what it leaves. What would have to be true of a street sequence for masking to help?**

The KITTI odometry benchmark has 22 sequences, from quiet suburbs to a motorway with traffic (sequence 01). They are a large download (the colour images are 65 GB in all), and pySLAM reads them directly: name the folder in `config.yaml` (`KITTI_DATASET`) and `pixi run slam` runs on the sequence it names; the segmentation script takes `--images path/to/sequences/NN/image_2 --pattern '*.png'`. Keep `is_color: True` there, so that SLAM reads the same colour images (`image_2`) that the label maps were made from. If you have the disk space and the time, look for a sequence on which masking makes an appreciable difference, and say why it does there.

7 Extra Activities

These are optional.

Activity 9: Your Own Video

If you ran SLAM on a video of your own in Lab 01, segment it and mask it the same way. The segmentation script takes the same `--video`, `--images` and `--tum` options as `slam`; the label maps are saved by frame number, so the two agree. Without ground truth there is no trajectory error, so look at what tracking does with and without the mask.